

# Selfless Traffic Routing with Multi-Agent PPO in SUMO: When Does Sacrificing Help?

Anh Vu

Department of Computer Science  
University of Houston

(I know it's 90th percentile, but it's good to speed out in the abstract)

**Abstract**—Most navigation systems route each vehicle independently along its own shortest path. When many vehicles make that same locally optimal choice, the shared roads they converge on become bottlenecks and total travel time rises—the gap between the selfish (Wardrop) equilibrium and the system optimum that traffic theory calls the price of anarchy. *Selfless routing* asks a subset of vehicles to accept a longer personal route when doing so lowers travel time for the whole fleet. We study whether a cooperative multi-agent reinforcement-learning policy can learn this behavior in a realistic simulator. We implement Multi-Agent Proximal Policy Optimization (MAPPO) with centralized training and decentralized execution: a shared per-candidate route actor chooses among up to four complete routes, and a centralized critic that sees a fleet-wide summary stabilizes learning. Cooperation is induced by a difference-reward team term measured directly in seconds of travel time, so the policy can weigh personal delay against fleet delay without an arbitrary conversion. We train the policy on a congested New York City corridor (450 controlled and 150 background vehicles routed to a shared destination) and evaluate it against a Dijkstra shortest-path baseline. On 20 held-out scenarios, the deployed greedy policy completes 100% of vehicles and reduces mean fleet travel time from 596.6 s to 559.2 s (−6.3%, winning 16/20 scenarios), with lower p90 travel time and fewer deadline misses; the improvement grows with congestion ( $r = -0.48$ ). An ablation of a congestion-aware detour guard reveals that the learned policy in fact prefers a non-shortest route 48% of the time: executing those detours lowers mean travel time further (−11.2%) by rescuing the most congested scenarios, yet hurts light-traffic scenarios where shortest paths are already near-optimal. The result is a concrete demonstration that the value of selfless routing tracks the network’s price of anarchy, and that a simple coordination guardrail trades peak fleet gains for robustness.

**Index Terms**—selfless traffic routing, multi-agent reinforcement learning, MAPPO, price of anarchy, SUMO, intelligent transportation systems

## I. INTRODUCTION

Traffic congestion is a system-level phenomenon, yet most routing systems attack it as a collection of independent shortest-path problems: each vehicle is guided along the route that is fastest for itself, taking the current network as fixed. When a large number of vehicles make that same locally optimal choice, the roads they share fill up, and the locally optimal path becomes a bottleneck that raises travel time for everyone. Classical traffic theory formalizes this as the gap between the *selfish* routing outcome—the Wardrop user equilibrium, in which no driver can improve their own time by switching routes [1]—and the *system optimum* that minimizes total travel time. The ratio between them is the *price of*

*anarchy*, and it can be strictly greater than one [2]; the Braess paradox is the extreme case, where adding capacity under selfish routing makes everyone slower [3].

The *selfless traffic routing* (STR) model of Dai et al. [4] proposes the constructive alternative: allow some controlled vehicles to accept a slightly longer route when that choice reduces the average travel time of the fleet, subject to individual constraints. This reframes routing as a cooperative decision problem rather than a per-vehicle optimization. The open question is whether a learned policy can discover *when* such sacrifices pay off—and by how much—in a realistic traffic environment rather than an idealized graph.

This paper studies that question with a cooperative multi-agent reinforcement-learning (MARL) controller in the Eclipse SUMO microscopic simulator [5]. We model each controlled vehicle as an agent that periodically chooses among a small set of complete candidate routes to its destination, and we train the shared policy with Multi-Agent Proximal Policy Optimization (MAPPO) [6], a centralized-training / decentralized-execution (CTDE) actor–critic method. Crucially, cooperation is not left to emerge on its own: each agent’s reward internalizes a share of the fleet’s congestion, expressed in *seconds of travel time*, so that the policy can trade its own delay against fleet delay on a common scale. We compare the learned controller against a Dijkstra shortest-path baseline [7] on the same generated demand.

### A. Research Question

*Can a cooperative MAPPO routing policy reduce fleet travel time relative to shortest-path routing, and under what traffic conditions do selfless (non-shortest) route choices actually help?*

The second half of the question is as important as the first: our results show that the answer depends sharply on how congested the scenario is.

### B. Contributions

- We formulate selfless routing as a cooperative Dec-POMDP over route choices and solve it with a CTDE MAPPO policy whose actor scores complete candidate routes and whose reward—including a *difference-reward* team term [8]—is expressed entirely in travel-time units, avoiding an arbitrary congestion-vs-time exchange rate.
- We evaluate the trained policy against Dijkstra on 20 held-out congested scenarios, reporting an honest, repro-



ducible outcome: the deployed policy reduces mean fleet travel time by 6.3% (16/20 wins) at 100% completion, with gains that grow with congestion.

- Through a guardrail ablation we show that the policy learns substantial selfless behavior (48% non-shortest route choices) and that executing those detours *helps* in heavy congestion but *hurts* in light traffic—a direct empirical link between the benefit of selflessness and the network’s price of anarchy.
- We document the failure modes that remain (a rare gridlock scenario, a weak learning signal on a low-price-of-anarchy grid) and the role of a congestion-aware coordination guardrail in trading peak gains for robustness.

## II. RELATED WORK

### A. Selfish routing and the price of anarchy

The study of routing as a game predates learning-based methods. Wardrop’s principles distinguish the user equilibrium, where every used route between an origin and destination has equal and minimal cost to the individual, from the system optimum that minimizes total cost [1]. Roughgarden and Tardos quantified how far apart these can be, proving bounds on the price of anarchy for selfish routing [2], and the Braess paradox shows the equilibrium can be counter-intuitively bad [3]. These results motivate selfless routing directly: the potential benefit of asking vehicles to sacrifice is exactly the price-of-anarchy gap, which is large only when the network forces contention onto scarce links. Our experiments make this dependence concrete—selfless detours help precisely in the saturated scenarios where the gap is wide, and not otherwise.

### B. The selfless traffic routing model

Dai et al. introduced the selfless driving model and argued that vehicular routing should be evaluated by network-level average travel time rather than individual shortest paths [4]. Their formulation permits a controlled vehicle to take a locally suboptimal route when doing so benefits the group. We adopt this objective but replace their heuristic controller with a learned cooperative policy, and we ground the trade-off in a common travel-time unit so the sacrifice decision is made on principled terms.

### C. Reinforcement learning and MAPPO

Reinforcement learning is a natural fit for sequential routing, where actions change the state that later actions face [9]. Proximal Policy Optimization (PPO) provides a stable on-policy actor-critic update via a clipped surrogate objective [10], and Generalized Advantage Estimation (GAE) reduces the variance of the advantage signal it optimizes [11]. In the multi-agent setting, the environment becomes non-stationary from any single agent’s view. The centralized-training / decentralized-execution paradigm addresses this by giving the critic access to global information during training while the actor remains local [12]. MAPPO applies PPO under CTDE with a centralized value function and has proven

surprisingly strong on cooperative benchmarks [6]; we use it here because our reward is cooperative and our deployment must be decentralized (each vehicle routes from its own view).

### D. Credit assignment in cooperative MARL

A cooperative reward shared identically by all agents makes it hard to tell which agent’s action helped, because each agent’s contribution is buried in a large common signal. Difference rewards address this by crediting each agent with its marginal effect on the global objective [8], and counterfactual multi-agent policy gradients (COMA) estimate that marginal effect with a centralized critic [13]. Our team-reward term is a leave-one-out difference reward in travel-time units: each agent pays for its own slowness relative to the live fleet average, so the exogenous, demand-driven congestion level—which no single vehicle controls—cancels out and the learnable signal is sharpened.

### E. Simulation substrate

We use Eclipse SUMO as the microscopic traffic environment [5], [14]. SUMO models car-following, lane changing, junctions, and traffic lights, which makes the evaluation considerably more realistic than routing on a static weighted graph, but also makes control harder: a route that looks good at the graph level is only useful if the vehicle can physically execute it under lane-feasibility, route-continuity, and commit-window constraints. Dijkstra’s algorithm [7] remains the appropriate baseline—not as a weak straw man, but as a strong, deterministic controller that respects the road graph and never over-commits, which is why it is competitive in exactly the low-congestion regime where selfless routing has nothing to gain.

## III. METHOD

### A. Problem Formulation

We control a fleet of  $N$  vehicles on a fixed SUMO road network while a background population of uncontrolled vehicles shares the roads. Each controlled vehicle  $i$  is an agent that, at a set of route-decision points along its trip, chooses a complete route to its destination. SUMO handles all low-level dynamics—car-following, lane changing, junctions, and signals—so the policy only sets high-level route intent. The fleet objective is to minimize average travel time,

$$J(\pi) = \frac{1}{N} \sum_{i=1}^N T_i, \quad (1)$$

where  $T_i$  is the realized travel time of vehicle  $i$  under the shared policy  $\pi$ . This is a cooperative, partially observed problem: each agent sees only a local view of the network but its route choice affects the congestion that other agents experience. We treat it as a decentralized POMDP and solve it with CTDE MAPPO, so that a fleet-wide signal can guide learning while deployment stays per-vehicle.

The tension the policy must resolve is precisely the price-of-anarchy gap of Section II: the route that minimizes  $T_i$  for one vehicle need not minimize  $J$  for the fleet. A useful policy



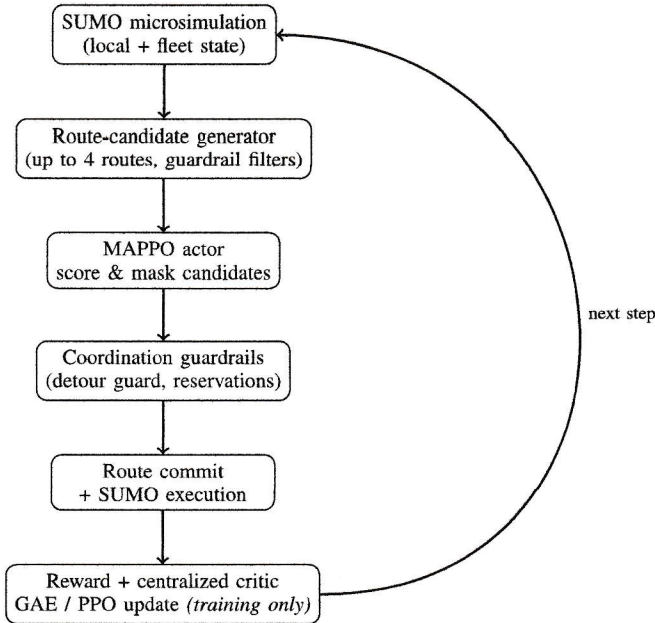


Fig. 1. The STR-SUMO learning and deployment pipeline. Every stage speaks a common travel-time / density vocabulary. The dashed portion (reward and critic) runs only during training; at deployment the actor and guardrails alone drive each vehicle.

learns when to keep a vehicle on its shortest feasible path (most of the time) and when to route it away from a forming bottleneck for the group’s benefit.

### B. Route-Candidate Action Space

Rather than asking the policy to pick a turn (“left/right”) at each junction, a route-candidate generator proposes up to  $K=4$  complete routes from the vehicle’s current edge to its destination. Index 0 is always the congestion-aware shortest route; indices 1–3 are diverse detours produced by density-penalized Dijkstra searches. The actor’s job is to choose one of these routes, so a “selfless detour” simply means choosing an index other than 0. Each candidate carries an 11-dimensional feature “scorecard” describing, among other things, its average and worst-case congestion, its first-edge congestion, the extra seconds it costs relative to the shortest route (the ETA sacrifice), and how much less congested it is than the shortest route (the relief). Because the reward, the actor, and the guardrails all read the same scorecard, a trade-off assessed by one component is understood identically by the others.

### C. CTDE Actor and Critic

Figure 2 shows the network. The **actor** is a shared per-candidate scorer: a small multilayer perceptron (two hidden layers of 256 and 128 units, ReLU) takes the vehicle’s local observation concatenated with one candidate’s features and emits a scalar score; the same sub-network scores all  $K$  candidates, and invalid candidates are masked to  $-10^9$  before a softmax over scores. This weight-sharing lets the actor generalize across a variable, unordered candidate set and keeps deployment per-vehicle. The local observation encodes the

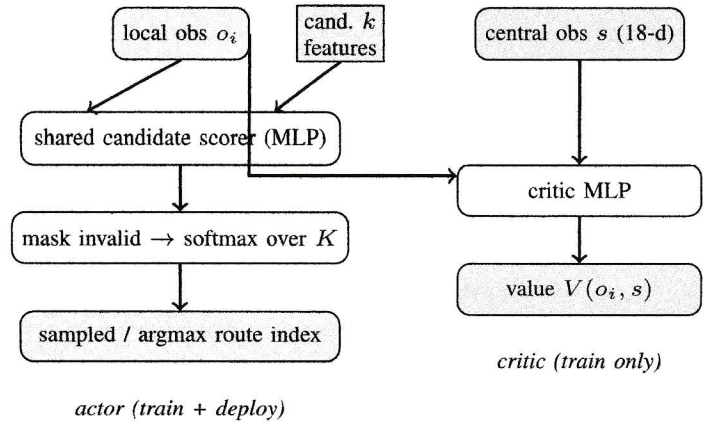


Fig. 2. CTDE MAPPO architecture. The actor scores each candidate route with a shared sub-network from the vehicle’s local view; the centralized critic additionally reads an 18-dimensional fleet summary and is discarded at deployment.

current and destination edges, feasibility masks, lane and commit-window state, route-difficulty estimates, local density summaries, and the per-candidate features.

The **critic** is used only during training. It concatenates the local observation with an 18-dimensional *central* observation summarizing the whole fleet’s state (aggregate densities, completion, congestion pressure) and predicts a scalar value. This is the CTDE split [12]: the critic “peeks” at global information to give lower-variance value estimates, while the actor never does, so the trained policy runs decentralized. The critic changes learning stability, not the objective; the cooperative incentive must come from the reward.

### D. Learning Algorithm

The policy is trained with MAPPO [6]: a clipped PPO surrogate for the actor [10] and a squared-error value loss for the critic, with an entropy bonus for exploration and separate Adam optimizers [15] for the two networks. Advantages are computed per vehicle trajectory with GAE [11] ( $\gamma=0.995$ ,  $\lambda=0.95$ ); the high discount is needed because trips span roughly two thousand simulation steps and a route choice must be credited for congestion it causes far downstream. Two practical components proved important on this task. First, because episode returns swing by more than an order of magnitude with the demand seed, the critic regresses on *normalized* value targets (running mean/variance), while advantages remain in real reward units; this keeps the value loss well-conditioned. Second, entropy is annealed from 0.15 to 0.05 over training so the policy explores detours early and settles later, and PPO updates are stopped early when the approximate KL divergence exceeds 0.015. Key hyperparameters are listed in Table I.

### E. Reward: Cooperation in Travel-Time Units

Every term of the reward is measured in seconds so trade-offs need no arbitrary conversion factor. Each simulation step charges a small base time cost (0.07 per step), plus deliberately



TABLE I  
MAPPO TRAINING CONFIGURATION

| Group        | Parameter                 | Value                                 |
|--------------|---------------------------|---------------------------------------|
| Actor/critic | hidden layers             | [256, 128], ReLU                      |
|              | candidate routes $K$      | 4                                     |
|              | central obs dim           | 18                                    |
| Optimization | actor / critic LR         | $3 \times 10^{-4} / 1 \times 10^{-3}$ |
|              | $\gamma$ / GAE $\lambda$  | 0.995 / 0.95                          |
|              | PPO clip $\epsilon$       | 0.05                                  |
|              | epochs / minibatch        | 4 / 512                               |
|              | target KL                 | 0.015                                 |
|              | value-target norm.        | on                                    |
| Exploration  | entropy coef. (start→end) | 0.15 → 0.05                           |
| Objective    | team-reward $\alpha$      | 1.0                                   |
|              | team-reward mode          | difference                            |

smaller costs for occupying above-average-density roads and for the congestion a vehicle imposes on others; reaching the destination pays +50 and a teleport (SUMO's last resort for a stuck vehicle) costs -40. The cooperative term is the key addition. Let  $\rho_i$  be vehicle  $i$ 's normalized slowness (0 at free flow, 1 at standstill) and  $\bar{\rho}$  the live-fleet mean. With weight  $\alpha \in [0, 1]$  and scale  $\kappa$ , agent  $i$  pays over an elapsed interval  $\Delta t$

$$c_i^{\text{team}} = \alpha \kappa (\rho_i - \bar{\rho}) \Delta t. \quad (2)$$

This is a leave-one-out difference reward [8]: because the fleet-mean of  $(\rho_i - \bar{\rho})$  is zero, the exogenous, demand-driven congestion level cancels, and each agent is credited only for how its own route choice fares relative to the fleet—a vehicle that routed onto a free-flowing alternative is rewarded, one that piled into a jam pays. We use  $\alpha=1$  (fleet delay counts as much as personal delay) with the difference mode throughout. Setting  $\alpha=0$  recovers a purely selfish objective and serves as a conceptual control.

#### F. Guardrails and the Saturation Veto

Hand-written guardrails keep the reward focused on travel time by removing obviously bad options rather than teaching basic safety through penalties. Pre-decision filters prune candidate routes that would loop, re-enter a dead end, or drive straight into a known jam; a “pending” state machine distinguishes healthy queueing (e.g. at a red light) from a genuine stall so the reward is not charged for normal waiting. Two coordination guardrails address a specific failure in heavy congestion, where independently chosen detours can pile onto the same alternative and make it worse:

- The **detour guard** (deployment only): after the actor picks a detour, if the alternative is itself near capacity and the network is congested, the choice is reverted to the shortest route. It judges the alternative by its *absolute* crowdedness, not by the relief the policy reacted to, because when everything is busy a road can look like relief yet still have no room left. It runs only at deployment, since overriding an executed route during training would break the on-policy assumption.
- The **route reservation** (training and deployment): a committed route books its leading edges in a decaying field,

so the candidate generator scores later deciders against effective (live + reserved) density and a simultaneous rush becomes an orderly one-at-a-time spread. It affects only what candidates look like, never the reward or the true measurements.

The detour guard is the subject of our ablation in Section V-D: it decides whether the policy's learned detours are executed or suppressed, and it turns out to trade peak fleet gains for robustness.

## IV. EXPERIMENTAL SETUP

### A. Network and Demand

Experiments use a New York City grid extracted for SUMO: 132 drivable edges, 360 lanes, 90 junctions (53 signalized), and roughly 27,000 lane-meters. To create genuine congestion—a precondition for selfless routing to have anything to gain—we use a shared-destination demand pattern (ranged origins routed to one common destination edge) with 450 controlled and 150 background vehicles released at a 0.5 s interval. This concentrates flow into a corridor and produces sustained congestion (frequent teleports, fleet travel times from a few hundred to over 1500 s across seeds) without permanent gridlock. A dispersed origin–destination pattern, by contrast, leaves the grid at a few percent of capacity, where routing cannot matter.

### B. Protocol, Baselines, and Metrics

The policy is trained for 100 episodes. Every 25 episodes it is evaluated frozen (no learning) on 11 held-out seeds (6000–6010) against a Dijkstra baseline generated from the identical demand, and the best checkpoint is retained. We then *deploy* the best checkpoint on a separate block of 20 held-out scenarios (seeds 4010–4029) and compare it head-to-head with Dijkstra on the same generated vehicles. We report mean fleet travel time, median (p50) and p90 travel time, tail-completion gap, the p95/p50 ratio, deadline misses, completion rate, and per-scenario win rate. Unless noted, the MAPPO policy is deployed *greedily* (argmax over candidate scores), which is deterministic and, as Section V-E shows, matches stochastic sampling once the detour guard is active. Each controller is run in a fresh simulation with independent network state, so no traffic leaks between runs.

## V. RESULTS AND DISCUSSION

### A. Training Dynamics

Figure 3 summarizes the 100-episode run. Episode outcomes are dominated by the demand realization, not by smooth policy improvement: fleet travel time swings from 266.8 s (episode 59) to 1889 s (episode 13) and correlates strongly with teleports ( $r=0.78$ )—the seeds that draw heavy demand gridlock regardless of routing (panels a,b). Completion stays high (mean 0.993). Underneath the noise the policy sharpens: entropy anneals from 0.99 to about 0.72 (panel c) and the mean margin between candidate scores grows from near zero to 1.08. ~~But~~ the PPO update magnitude stays small—the approximate KL divergence has a median of only 0.0015 against a 0.015

However,



TABLE II  
DEPLOYMENT OVER 20 HELD-OUT SCENARIOS (SEEDS 4010–4029)

| Metric               | Dijkstra | MAPPO<br>(guard on) | MAPPO<br>(guard off) |
|----------------------|----------|---------------------|----------------------|
| Completion rate      | 1.000    | 1.000               | 1.000                |
| Avg. travel time (s) | 596.6    | 559.2               | <b>529.8</b>         |
| vs. Dijkstra         | –        | –6.3%               | –11.2%               |
| p50 travel time (s)  | 576.7    | 541.9               | <b>525.9</b>         |
| p90 travel time (s)  | 1097.6   | 1061.8              | <b>965.5</b>         |
| Tail gap (steps)     | 596.7    | 587.7               | <b>540.9</b>         |
| p95/p50 ratio        | 2.17     | 2.24                | <b>2.05</b>          |
| Deadline misses      | 225.8    | <b>213.6</b>        | 216.8                |
| Non-shortest rate    | –        | 0.00                | 0.48                 |
| Scenarios won        | –        | <b>16/20</b>        | 12/20                |

target and never trips early stopping (panel d)—so the per-update learning signal is weak. This is the expected signature of a low-price-of-anarchy environment: with many near-equal parallel routes on a grid, the advantage of any single route choice is small and easily buried by exogenous traffic variance. Value-target normalization keeps the critic well-conditioned throughout (median value loss 0.18). Because later episodes are not reliably better, we select the deployment checkpoint by held-out evaluation rather than by final weights.

### B. Checkpoint Selection

Held-out evaluation on seeds 6000–6010 selected the episode-24 checkpoint. There it completes all vehicles and reduces mean fleet travel time from 597.3 s (Dijkstra) to 537.0 s (–10.1%), winning 8 of 11 scenarios, with lower p90 (1035 s vs 1071 s) and fewer deadline misses (208 vs 220). Notably, the deployed greedy policy takes *zero* non-shortest routes on these seeds—its advantage over Dijkstra comes entirely from recomputing the shortest route against live congestion, not from visible detours. This foreshadows the central finding: on this network the fleet benefit and the amount of overt sacrifice are largely decoupled.

### C. Deployment: MAPPO vs. Dijkstra

We deploy the selected checkpoint on 20 held-out scenarios (seeds 4010–4029) against Dijkstra on identical demand (Table II, Fig. 4). Both complete 100% of vehicles. Greedy MAPPO—with the detour guard active, our default deployment—reduces mean fleet travel time from 596.6 s to 559.2 s (–6.3%, median per-scenario –22.3 s), winning 16 of 20 scenarios, and improves p90 travel time (1062 s vs 1098 s) and deadline misses (214 vs 226). The improvement grows with congestion (correlation between per-scenario gain and Dijkstra travel time  $r = -0.48$ , Fig. 4b): the largest wins are the most congested scenarios—seed 4028 (1109 → 713 s) and seed 4010 (1585 → 1249 s)—while the few losses are light-traffic scenarios where the shortest path is already near-optimal (seeds 4014, 4013 at +8 s each). One scenario, seed 4012, is a genuine failure: MAPPO nearly doubles travel time (664 → 1191 s), a rare gridlock we return to in Section V-F.

However,

### D. When Do Selfless Detours Help? A Detour-Guard Ablation

The default deployment executes zero overt detours, yet the underlying policy is far from passive. The *detour guard* is a deployment-time filter that keeps a vehicle on the shortest route when its chosen alternative is already near capacity and the network is congested. Removing it reveals that the learned policy in fact chooses a non-shortest route 48% of the time (Table II, right column). Executing those detours lowers mean fleet travel time *further*, to 529.8 s (–11.2% vs. Dijkstra), and improves p90 (965.5 s) and the p95/p50 spread (2.05), because spreading load across alternatives thins the tail. **But** the gain is concentrated entirely at high congestion (Fig. 4c): the detours rescue the congested scenarios (seed 4012 1191 → 838 s, seed 4010 1249 → 1030 s) while *hurting* light-traffic scenarios, where a longer route only adds distance with no jam to relieve (seeds 4016, 4020, 4014 each about +75 s). The net effect is that the unguarded policy wins fewer scenarios (12/20) despite a better mean and tail.

This is the paper’s central result. The value of selfless routing is not a fixed property of the policy; it tracks the network’s price of anarchy. Where contention is high, the learned detours move vehicles off congested corridors onto genuine spare capacity and cut fleet travel time substantially; where the grid is lightly loaded, selfish shortest-path routing already approximates the system optimum and every detour is wasted distance. The detour guard encodes exactly this distinction as a deployment-time robustness knob: suppressing detours in heavy congestion converts the higher-mean-gain policy into a safer one that wins more scenarios but forgoes the largest fleet improvements.

### E. Greedy vs. Stochastic Deployment

With the detour guard active, greedy (argmax) and stochastic (sampled) deployment produce identical results on all 20 scenarios. Two effects combine: the trained policy’s candidate scores are sharply peaked (mean margin 1.08), so sampling rarely diverges from the argmax, and any detour sampling does select is reverted by the guard in this congested regime. Route diversity—the usual argument for stochastic deployment as a load-spreading mechanism—is therefore delivered here by the route reservation field and the policy itself rather than by sampling noise. When the guard is removed the two modes diverge, but so does the rare gridlock behavior that independent sampling can cause when many vehicles commit to the same corridor at once.

### F. Limitations

Three limitations qualify these results. First, the aggregate gains are driven by a minority of severe-congestion scenarios; this is arguably the point of selfless routing, but it inflates means and widens variance. Second, one scenario (seed 4012) is a true failure in which the greedy policy gridlocks and the guard does not prevent it, indicating that the coordination guardrails are necessary but not sufficient. Third, the weak per-update learning signal (Section V-A) means the policy is likely under-trained relative to its ceiling; the low price of



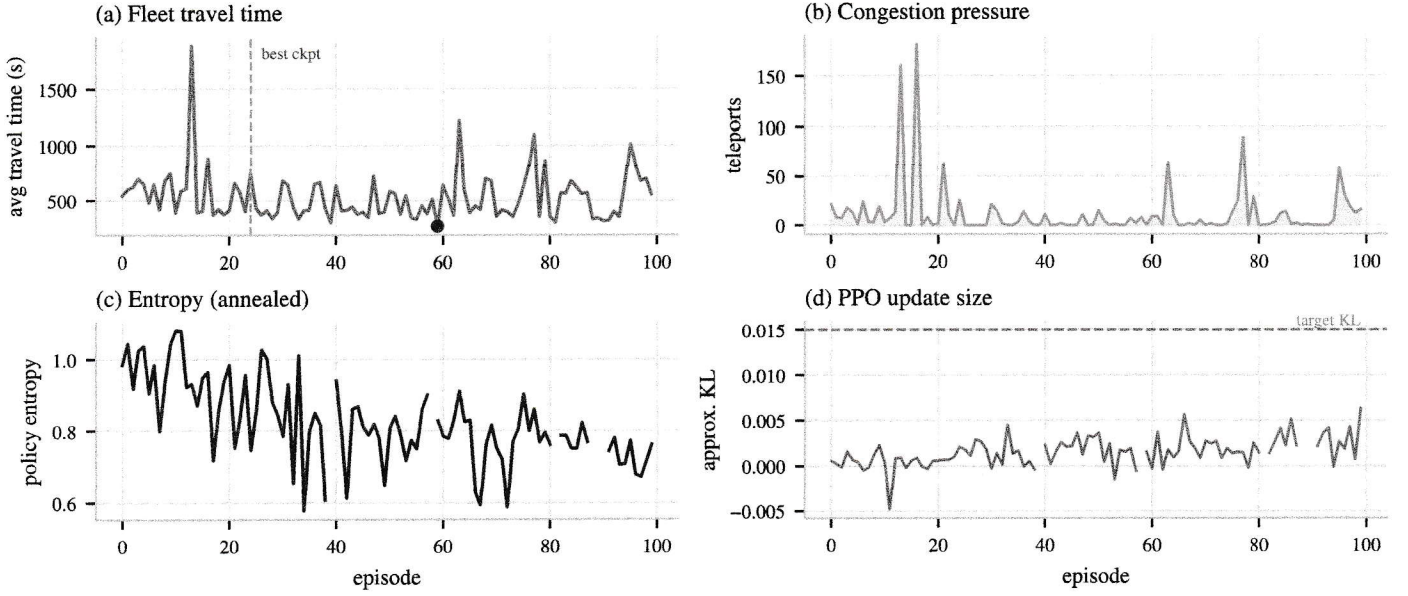


Fig. 3. MAPPO training over 100 episodes on the congested corridor. (a) Fleet travel time is highly variable and (b) tracks congestion spikes (teleports): episode outcomes are demand-driven. (c) Policy entropy anneals as the policy sharpens, but (d) PPO updates stay well below the target KL, indicating a weak per-update learning signal—the expected behavior on a low-price-of-anarchy grid.

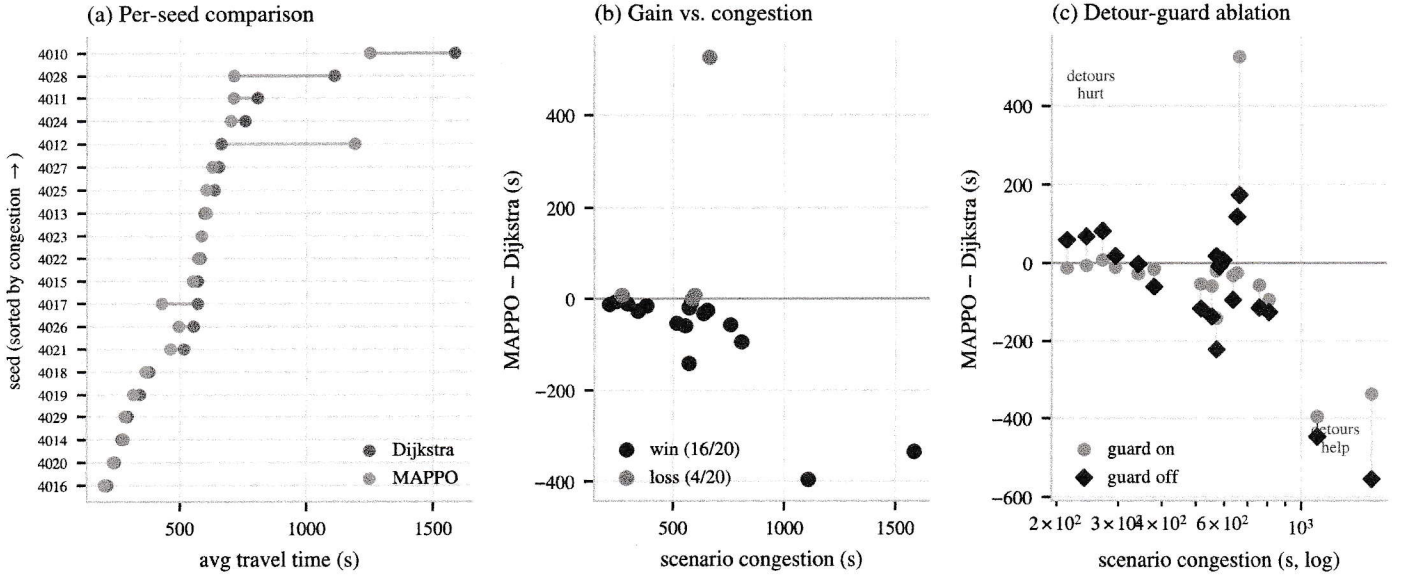


Fig. 4. Deployment on 20 held-out scenarios. (a) Per-scenario fleet travel time, Dijkstra vs. greedy MAPPO, sorted by congestion; green connectors mark MAPPO wins, red losses. (b) The per-scenario gain grows with congestion ( $r = -0.48$ ): MAPPO’s largest improvements are on the most congested scenarios, with the few losses at light traffic and one gridlock outlier (seed 4012). (c) Removing the detour guard lets the policy execute its 48% detours (violet): these hurt in light traffic (points above zero, left) but help increasingly as congestion rises (points below the guarded orange, right).

anarchy of a dense grid limits how much any routing policy can gain, which both explains the modest headline number and motivates evaluation on higher-price-of-anarchy networks. We report these openly because the honest contribution is the *conditions* under which selfless routing helps, not a single benchmark number.

## VI. CONCLUSION AND FUTURE WORK

We studied selfless traffic routing with a cooperative MAPPO policy in SUMO, using a centralized critic, a shared

per-candidate route actor, and a difference-reward team objective expressed in seconds of travel time. On a congested New York City corridor, the learned policy is a genuine improvement over shortest-path routing: deployed greedily on 20 held-out scenarios, it completes every vehicle and cuts mean fleet travel time by 6.3% (16/20 wins), with a lower tail and fewer deadline misses, and its advantage grows with congestion.

The more interesting finding is conditional. The policy learns to take non-shortest routes almost half the time, and whether executing those routes helps depends on the traffic

regime: in heavy congestion the detours relieve real bottlenecks and lower fleet travel time to 11.2% below Dijkstra, but in light traffic they only add distance. Selfless routing pays in proportion to the network’s price of anarchy—the very quantity that traffic theory uses to bound the cost of selfishness. The detour guard turns this insight into a deployment knob that trades peak fleet gains for per-scenario robustness.

Several directions follow. First, and most directly motivated by our data, is evaluation on high-price-of-anarchy networks—Pigou- or Braess-style bottlenecks where the selfish equilibrium is provably far from optimal—so that selfless routing has a large and persistent margin to capture rather than the intermittent one a dense grid offers. Second, the weak per-update learning signal invites stronger credit assignment, such as a counterfactual critic [13] or explicit marginal-cost congestion pricing, to make the fleet externality visible to each agent’s advantage. Third, the single gridlock failure (seed 4012) shows the coordination guardrails need an adaptive, rather than fixed-threshold, congestion response. Finally, replacing the per-edge state encoding with a graph neural network would let a policy trained on one network transfer to others, moving selfless routing from a single-map demonstration toward a deployable controller.

## REFERENCES

- [1] J. G. Wardrop, “Some theoretical aspects of road traffic research,” *Proceedings of the Institution of Civil Engineers*, vol. 1, no. 3, pp. 325–362, 1952.
- [2] T. Roughgarden and É. Tardos, “How bad is selfish routing?” *Journal of the ACM*, vol. 49, no. 2, pp. 236–259, 2002.
- [3] D. Braess, A. Nagurney, and T. Wakolbinger, “On a paradox of traffic planning,” *Transportation Science*, vol. 39, no. 4, pp. 446–450, 2005.
- [4] G. Dai, P. K. Paluri, T. Carmichael, A. M. K. Cheng, and R. Mikkulainen, “Work-in-progress: Leveraging the selfless driving model to reduce vehicular network congestion,” in *2019 IEEE Real-Time Systems Symposium (RTSS)*, 2019, pp. 548–551.
- [5] P. A. Lopez, M. Behrisch, L. Bieker-Walz, J. Erdmann, Y.-P. Flötteröd, R. Hilbrich, L. Lücken, J. Rummel, P. Wagner, and E. Wießner, “Microscopic traffic simulation using SUMO,” in *2018 21st International Conference on Intelligent Transportation Systems (ITSC)*, 2018, pp. 2575–2582.
- [6] C. Yu, A. Velu, E. Vinitsky, J. Gao, Y. Wang, A. Bayen, and Y. Wu, “The surprising effectiveness of PPO in cooperative multi-agent games,” *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, vol. 35, pp. 24 611–24 624, 2022.
- [7] E. W. Dijkstra, “A note on two problems in connexion with graphs,” *Numerische Mathematik*, vol. 1, no. 1, pp. 269–271, 1959.
- [8] D. H. Wolpert and K. Tumer, “Optimal payoff functions for members of collectives,” *Advances in Complex Systems*, vol. 4, no. 2–3, pp. 265–279, 2001.
- [9] V. Mnih, K. Kavukcuoglu, D. Silver, A. A. Rusu, J. Veness, M. G. Bellemare, A. Graves, M. Riedmiller, A. K. Fidjeland, G. Ostrovski, S. Petersen, C. Beattie, A. Sadik, I. Antonoglou, H. King, D. Kumaran, D. Wierstra, S. Legg, and D. Hassabis, “Human-level control through deep reinforcement learning,” *Nature*, vol. 518, no. 7540, pp. 529–533, 2015.
- [10] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, “Proximal policy optimization algorithms,” *arXiv preprint arXiv:1707.06347*, 2017.
- [11] J. Schulman, P. Moritz, S. Levine, M. Jordan, and P. Abbeel, “High-dimensional continuous control using generalized advantage estimation,” in *International Conference on Learning Representations (ICLR)*, 2016.
- [12] R. Lowe, Y. Wu, A. Tamar, J. Harb, P. Abbeel, and I. Mordatch, “Multi-agent actor-critic for mixed cooperative-competitive environments,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017.
- [13] J. Foerster, G. Farquhar, T. Afouras, N. Nardelli, and S. Whiteson, “Counterfactual multi-agent policy gradients,” in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 32, no. 1, 2018.
- [14] D. Krajzewicz, J. Erdmann, M. Behrisch, and L. Bieker, “Recent development and applications of SUMO – simulation of urban mobility,” *International Journal on Advances in Systems and Measurements*, vol. 5, no. 3–4, pp. 128–138, 2012.
- [15] D. P. Kingma and J. Ba, “Adam: A method for stochastic optimization,” in *International Conference on Learning Representations (ICLR)*, 2015.

## APPENDIX: REPRODUCIBILITY

The SUMO network, demand-generation code, MAPPO implementation, trained checkpoints, and the training and evaluation CSV logs used in this paper are available at:

[https://github.com/anhvu0/STR-Summer-Project-Base/tree/test\\_branch\\_mappo](https://github.com/anhvu0/STR-Summer-Project-Base/tree/test_branch_mappo)

Training logs are in `configurations/rl_episode_metrics` and `configurations/rl_frozen_eval_metrics.csv`; the deployment comparison is reproduced with `main.py` on seeds 4010–4029.